

AGH University of Krakow
Faculty of Computer Science, Electronics and Telecommunication
Institute of Electronics

Systemy dedykowane w układach programowalnych

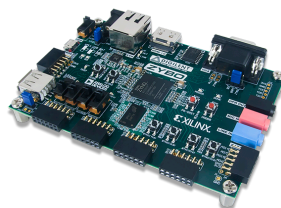
Filtracja sygnału audio w FPGA

Paweł Olbrych, Michał Zelek

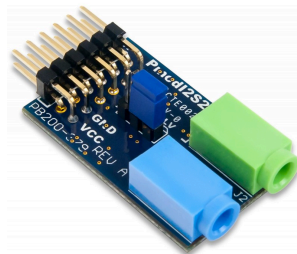
Spis treści

- [Cel projektu](#)
- [Teoria filtracji](#)
- [Opis sprzętowy](#)
- [Integracja z PYNQ](#)
- [Testy systemu](#)

Cel projektu Celem projektu było stworzenie systemu filtracji sygnału audio w oparciu o układ FPGA. Filtracja dźwięku została zrealizowana jako kaskadowe połączenie ośmiu filtrów IIR. Projekt wykorzystuje płytke Zybo z układem XC7Z010 i moduł Pmod I2S2. Logika programowalna odpowiada m.in. za obsługę modułu Pmod I2S2, który przesyła dane audio w standardzie I2S o formacie 24-bitowym z częstotliwością próbkowania 48 kHz. Bloki konfigurowalne odpowiadają także za implementację filtrów IIR oraz sterowanie torem przetwarzania. Projekt wykorzystuje układ ZYNQ, który za pomocą magistrali AXI pozwala użytkownikowi sterować torem audio oraz wgrywać własne współczynniki do banku filtrów. Procesor zapewnia także dostęp do wejściowych i wyjściowych próbek audio, które są buforowane przez FIFO, a następnie przesyłane do pamięci z użyciem mechanizmu DMA. System został opisany z wykorzystaniem VHDL w środowisku Vivado 2022.2.

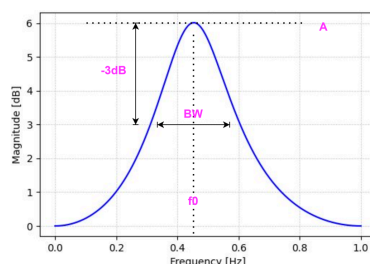


Płytką Zybo.



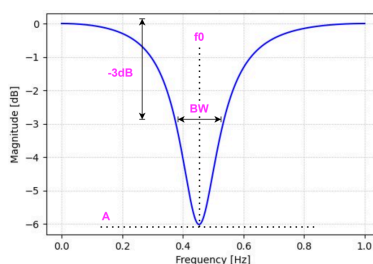
Moduł Digilent I2S2.

Teoria Filtrację dźwięku można zrealizować jako **bank filtrów** — zestaw filtrów połączonych równolegle lub szeregowo i różniących się np. częstotliwością środkową czy typem. System taki pozwala kształtować charakterystykę częstotliwościową sygnału stosownie do potrzeb. - **Filtr peak (band-pass/boost-cut)** służy do zwiększenia poziomu amplitudy sygnału w zadanym paśmie.



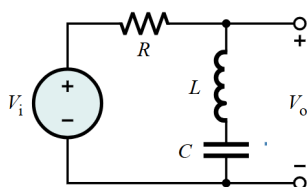
Charakterystyka filtru peak (oprac. własne).

- **Filtr notch (band-stop)** tłumi wybrane składowe częstotliwościowe.



Charakterystyka filtru notch (oprac. własne).

Parametry filtrów **peak** i **notch**: - f_0 – częstotliwość środkowa - A – wartość wzmocnienia - BW – pasmo (szerokość mierzona przy $A-3$ dB; dla notch zawsze -3 dB) Obydwa są filtrami IIR 2-go rzędu, charakteryzującymi się dużą selektywnością. ### Projekt filtru IIR W procesie projektowania często korzysta się z prototypu analogowego, np. filtru LC z rys. 3, wyznaczając wartość transmitancji układu w dziedzinie operatora Laplace'a. Następnie z użyciem transformacji biliniowej następuje przejście do dziedziny z i ponowne wyznaczenie transmitancji. Przekształcając transmitancję otrzymujemy równanie różnicowe, które opisuje działanie filtru IIR drugiego rzędu i bezpośrednio jest używane w jego implementacji sprzętowej/programowej.



filtr pasmowo-zaporowy

s Transmitancja obwodu:

$$H(s) = \frac{V_o}{V_i}, \quad s = j2\pi f = j\omega.$$

Impedancje elementów:

$$X_C = \frac{1}{j\omega C} = \frac{1}{sC}, \quad X_L = j\omega L = sL.$$

Stosując prawo Kirchhoffa otrzymujemy

$$\frac{V_i - V_o}{R} = \frac{V_o}{X_L + X_C}.$$

Po uproszczeniu:

$$H(s) = \frac{s^2 + \frac{1}{LC}}{s^2 + s\frac{R}{L} + \frac{1}{LC}} = \frac{s^2 + \omega_0^2}{s^2 + s\omega_w + \omega_0^2},$$

gdzie ω_0 – pulsacja środkowa, ω_w – pulsacja pasma. ### Transformacja biliniowa

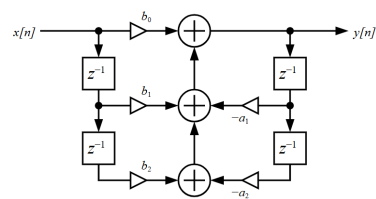
$$s = \frac{2}{T} \frac{z - 1}{z + 1}$$

pozwala przejść do dziedziny cyfrowej:

$$H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}} = \frac{Y(z)}{X(z)}.$$

Równanie różnicowe (DF-I):

$$y[n] = b_0 x[n] + b_1 x[n-1] + b_2 x[n-2] - a_1 y[n-1] - a_2 y[n-2].$$



Filtr IIR.

Współczynniki filtrów peak / notch

Współczynnik	Wzór
a_0	$4 + 2\frac{g}{Q} \omega_c T + (\omega_c T)^2$
a_1	$\frac{2((\omega_c T)^2 - 8)}{a_0}$
a_2	$\frac{4 - 2\frac{g}{Q} \omega_c T + (\omega_c T)^2}{a_0}$

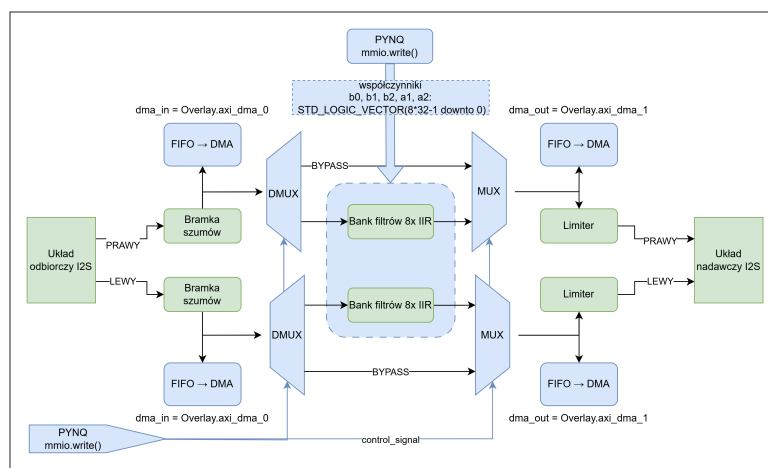
Współczynnik	Wzór
--------------	------

b_0	$\frac{4 + 2\frac{1}{Q}\omega_c T + (\omega_c T)^2}{a_0}$
-------	---

b_1	$\frac{-2((\omega_c T)^2 - 8)}{a_0}$
-------	--------------------------------------

b_2	$-\frac{4 - 2\frac{1}{Q}\omega_c T + (\omega_c T)^2}{a_0}$
-------	--

Parametry: * g – liniowa wartość wzmocnienia * Q – dobroć filtra * T – okres próbkowania * ω_c – pulsacja środkowa ## Opis sprzętowy Schemat poniżej przedstawia podział architektury wewnętrznej systemu na osobne moduły funkcjonalne.



Architektura systemu.

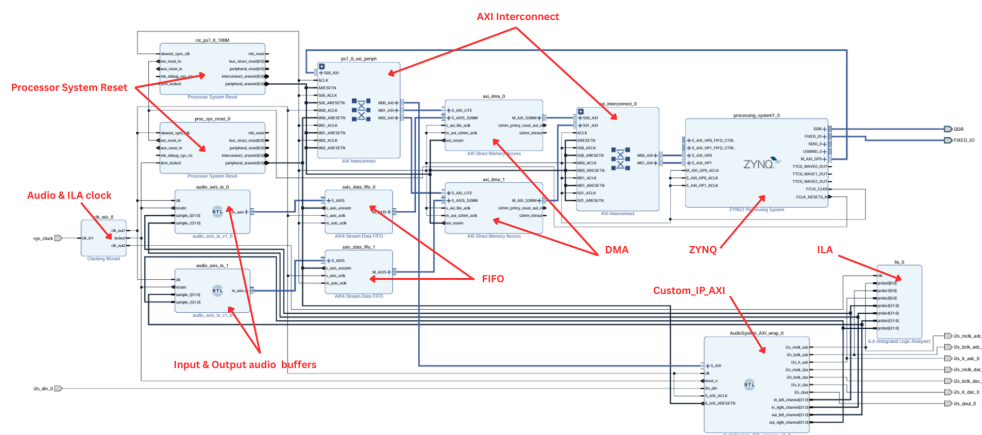
Moduły:

- **Moduł nadawczo/odbiorczy I2S** - obsługa modułu Pmod I2S2, serializacja/deserializacja danych audio
- **Bramka szumów** - moduł komparatora, który przekazuje dane audio do toru przetwarzania jeśli przekroczyły one ustalony próg, dane poniżej progu (tj. -60dB) traktowane są jako szum
- **Układ sterowania torem przetwarzania** - moduł składający się z demultiplekserów i multiplekserów, które za pomocą sygnału sterującego `control_signal` pozwalają na wybór aktywnej ścieżki audio
- **Bank filtrów** - połączenie kaskadowe ośmiu filtrów IIR drugiego rzędu
- **Limiter** - układ tłumiący wyjściowy sygnał audio, który przekroczył ustalony próg. Zadaniem limitera jest eliminacja przesterowań
- **FIFO->DMA** - bloki zewnętrzne IP Core, które buforują dane z wejścia i wyjścia sygnału audio i przesyłają do pamięci przez DMA (sygnał wejściowy - `dma_0`, sygnał wyjściowy

- dma_1)

Projekt blokowy

Zdjęcie poniżej przedstawia **Block Design** układu w Vivado



Block Design.

Układ oznaczony na rysunku wyżej jako **Custom_IP_AXI** stanowi główny moduł projektu, a jego budowę wewnątrz przedstawiono wyżej na Rys. x.

Diagram składa się z:

- **Audio & ILA clock** - generacja zegarów 12.288 MHz do obsługi Pmod I2S2 i 100MHz do modułu ILA
- **Input & Output audio buffers** - bliźniacze moduły które zbierają dane z wejścia i wyjścia audio dla obu kanałów i przesyłają je przez AXI do FIFO
- **ILA** - analizator wykorzystywany do testów
- **Processor System Reset**
- **AXI Interconnect**
- **DMA**
- **FIFO**
- **ZYNQ**

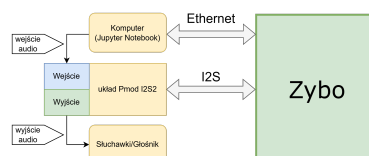
Podłączenie układu

- Fizyczne podłączenie układu:



Widok fizyczny projektu (oprac. własne).

- Schemat blokowy połączeń:



Schemat blokowy połączeń fizycznych (oprac. własne).

Integracja z PYNQ

Wszystkie funkcje służące do inicjalizacji czy wizualizacji wyników zostały umieszczone w utworzonej bibliotece - plik `functions.py` w katalogu `tools`.

Poniższy blok ładuje Overlay z wyeksportowanego w Vivado pliku `.xsa` oraz załącza bibliotekę `tools`

```
In [1]: from pynq import Overlay
from tools.functions import * # custom library with functions declarations
zybo_sdup_ov = Overlay("audio_AXI_DMA_ILA_48k_wrapper.xsa")
```

Sprawdzenie czy Overlay został poprawnie załadowany

```
In [2]: zybo_sdup_ov?
```

Utworzenie obiektu MMIO dzięki któremu uzyskujemy dostęp do rejestrów `slv` zdefiniowanych w AXI wrapper

```
In [2]: ip_name = "AudioSystem_AXI_wrap_0"
ip_info = zybo_sdup_ov.ip_dict[ip_name]
print(ip_info["phys_addr"])

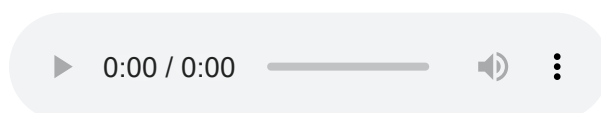
mmio = init_mmio(ip_name, zybo_sdup_ov)
load_mmio(mmio)
```

1073741824

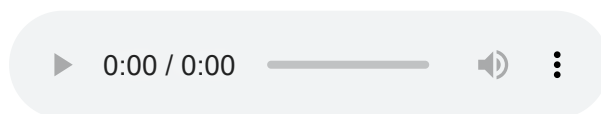
Próbki audio

Poniżej umieszczono 3 odtwarzacze z przygotowanymi utworami audio, które mogą być wykorzystane do przetestowania filtrów zdefiniowanych w rozdziale poniżej [Demonstracja filtracji](#)

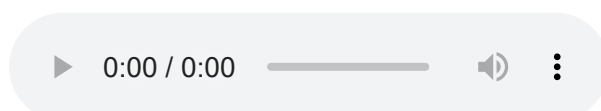
- Muzyka bez zniekształceń



- Muzyka z sinusoidalnym zniekształceniem o częstotliwości 5kHz



- Muzyka z sinusoidalnym zniekształceniem o częstotliwości 2kHz



Demonstracja filtracji

Blok poniżej, wgrywa współczynniki do banku filtrów w konfiguracji w której wzmacniane są niskie częstotliwości a tłumione wysokie

```
In [3]: # kaskada filtrów wzmacniających niskie częstotliwości
load_filters_from_json("filters/bass_enhancer_filters.json", mmio, enable_eq=True)
```

Blok poniżej, wgrywa współczynniki do banku filtrów w konfiguracji w której wzmacniane są wysokie częstotliwości a tłumione niskie

```
In [4]: # kaskada filtrów tłumiących niskie częstotliwości
load_filters_from_json("filters/bass_attenuate_filters.json", mmio, enable_eq=True)
```

Odtwarzając dźwięk z zniekształceniem sinusoidalnym o częstotliwości 2kHz, po uruchomieniu polecenia poniżej można zaobserwować wytłumienie zakłócenia

```
In [ ]: # kaskada filtrów tłumiących częstotliwość 2kHz z Q=5 => pasmo=400Hz
load_filters_from_json("filters/2k_attenuate_filters_narrow.json", mmio, enable_eq=True)
```

Odtwarzając dźwięk z zniekształceniem sinusoidalnym o częstotliwości 5kHz, po uruchomieniu polecenia poniżej można zaobserwować wytłumienie zakłócenia, jednak szerokie pasmo filtru wynoszące 5kHz, zauważalnie tłumi sąsiednie częstotliwości z zakresu [2.5kHz - 7.5kHz]

```
In [5]: # kaskada filtrów tłumiących częstotliwość 5kHz z Q=1
load_filters_from_json("filters/5k_attenuate_filters.json", mmio, enable_eq=True)
```

Użycie filtru z większym współczynnikiem dobroci $Q=5$, tworzy filtr z pasmem 1kHz, co zauważalnie zmniejszyło tłumienie sąsiednich częstotliwości - większa selektywność filtru

```
In [6]: # kaskada filtrów tłumiących częstotliwość 5kHz z większą selektywnością
# tj.  $Q=5$ ,  $\Rightarrow$   $\text{pasmo filtru} = f_{\text{filtra}}/Q \Rightarrow \text{Pasma} = 5000/5 = 1000\text{Hz}$ 
load_filters_from_json("filters/5k_attenuate_filters_narrow.json", mmio, enable_eq=True)
```

Wyłączenie filtrów

Uruchomienie poniższego bloku spowoduje przełączenie toru audio w tryb BYPASS - dźwięk trafi na wyjście z ominięciem toru przetwarzania

```
In [28]: #BYPASS brak filtracji sygnału
bypass_mode(mmio)
```

Dynamiczne sterowanie filtrami

Suwaki poniżej pozwalają na zaprojektowanie filtru IIR (notch/peak) o żądanych parametrach:

- częstotliwości środkowej **f_c**

- współczynnik dobroci $Q = fc/\text{Bandwidth}$ - określa selektywność filtru im większe Q tym większa selektywność
- wzmocnienia **Gain** podanego w decybelach

Dla ustalonych parametrów zostaną obliczone współczynniki, które zostaną wgrane do każdego z 8 filtrów IIR w banku filtrów po wciśnięciu przycisku "**Apply**"

In [9]: `display(ui)`

```
VBox(children=(IntSlider(value=2000, continuous_update=False, description='fc [Hz]', max=20000, min=16, style=...
```

Odbiór danych z DMA

Przygotowana konfiguracja pozwala na odczyt z DMA wartości wejściowych próbek audio (przed torem przetwarzania) oraz wyjściowych próbek (za torem przetwarzania) dla kanałów R i L

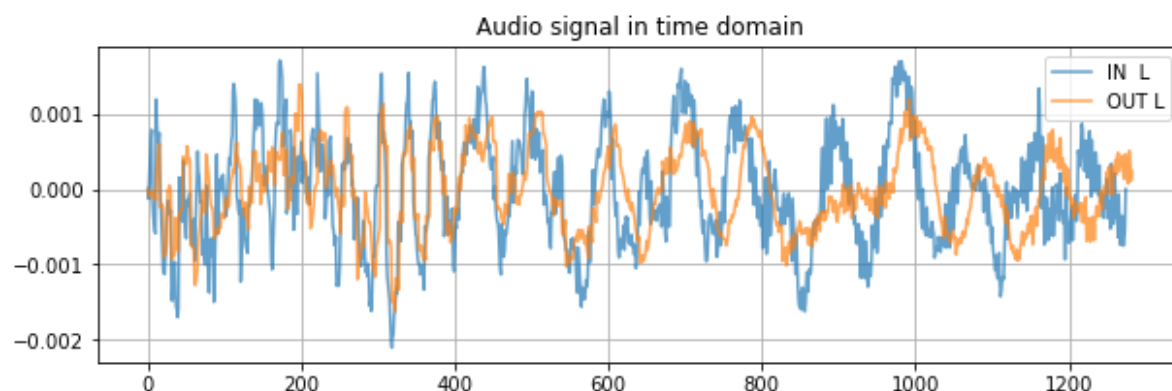
```
In [10]: #initialize DMA
dma_in, dma_out = init_dma(zybo_sdup_ov)
#capture data from in and out for both channels
left_in, right_in, left_out, right_out = capture_audio_from_channels(dma_in, dma_out)
#remove duplicates due to oversampling
left_in, right_in = delete_duplicates(left_in, right_in)
left_out, right_out = delete_duplicates(left_out, right_out)
# Convert data to floating point
audio_signals = (left_in, right_in, left_out, right_out)
audio_signals = [fixed_point_to_float(data_buffer) for data_buffer in audio_signals]
left_in_f, right_in_f, left_out_f, right_out_f = audio_signals
```

Captured 262144 words = 2048.0 kB

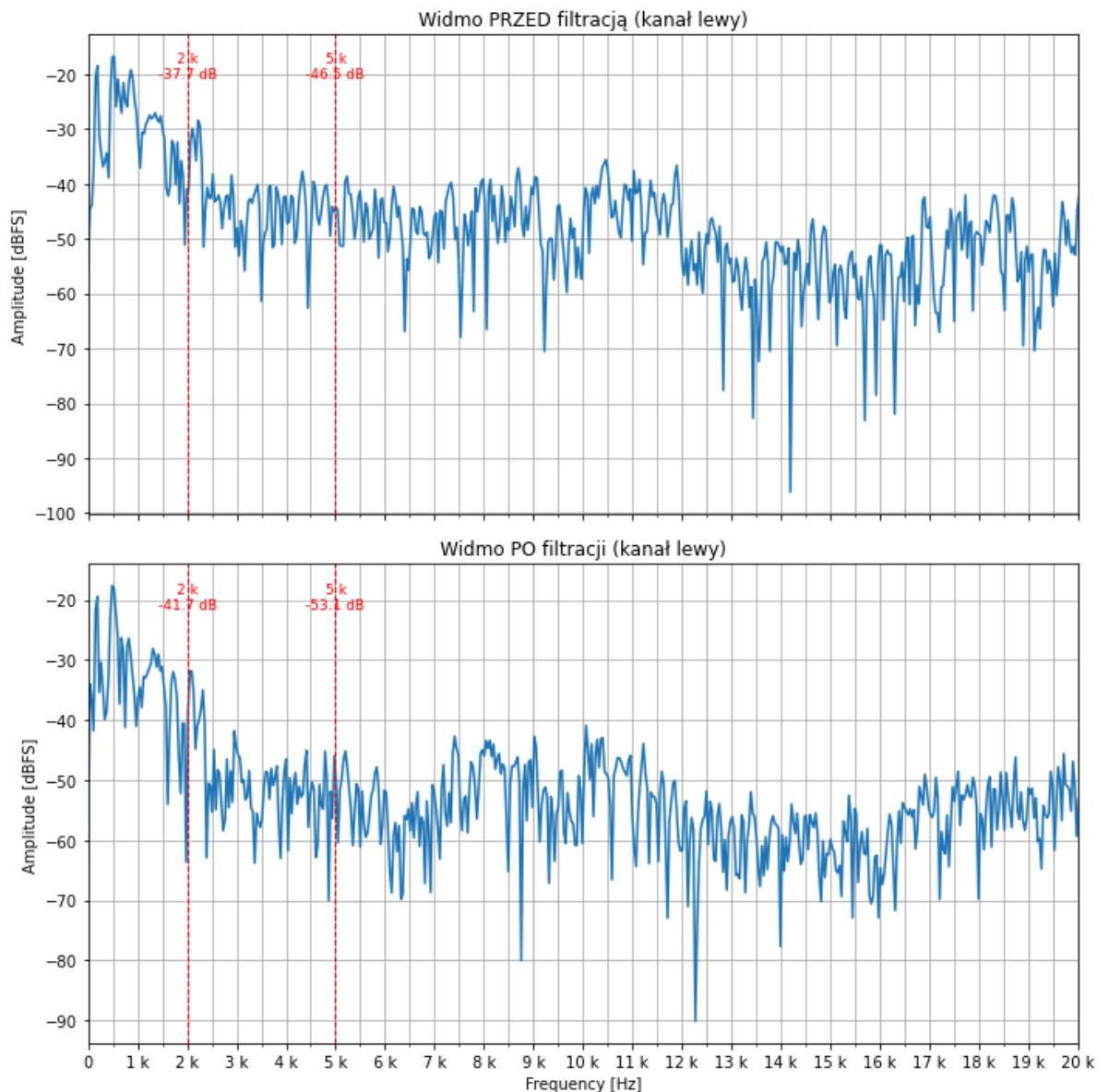
AFTER DUPLICATES REJECTION: 1275 SAMPLES IN ; 1275 OUT

AFTER DUPLICATES REJECTION: 1283 SAMPLES IN ; 1283 OUT

In [11]: `plot_audio_in_vs_out_time_domain(left_in_f, left_out_f)`



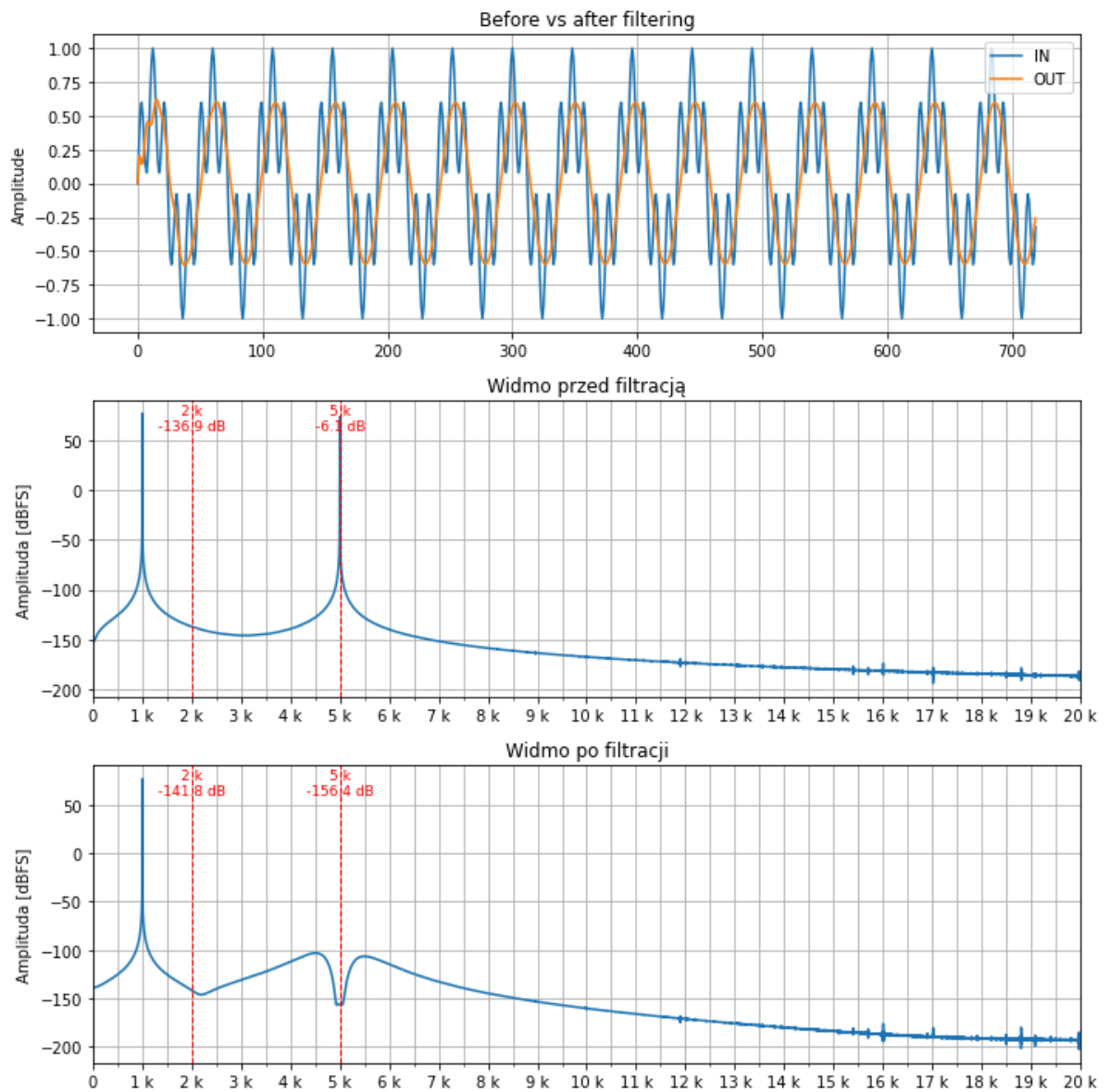
In [12]: `signal_in_f_domain, hardware_out_f_domain = plot_audio_in_vs_out_freq_domain(left_in_f, left_out_f)`



Filtracja realizowana programowo

Poniżej przedstawiono realizację kaskadowego połączenia filtrów IIR programowo. Funkcja `filter_and_plot()` przyjmuje jako parametr plik wejściowy JSON na podstawie którego tworzy kaskadowe połączenie filtrów. Sygnał testowy do filtrów składa się z dwóch sinusoid o częstotliwościach 1kHz i 5kHz. Wgrana konfiguracja filtrów tłumi sygnał o częstotliwości 5kHz.

```
In [13]: import numpy as np
fs = 48_000
t = np.arange(0, 1.0, 1 / fs)
test_sig_in = 0.6 * np.sin(2 * np.pi * 1_000 * t) + 0.4 * np.sin(2 * np.pi * 5_000
_, _, dt = filter_and_plot("filters/5k_atteunate_filters_narrow.json", test_sig_in)
```



Filtering time: 27.39 ms

Poniższy blok prezentuje czas filtracji przez filtr pojedynczej próbki, co symuluje process pipelining i pozwala określić opóźnienie jakie jest wprowadzane

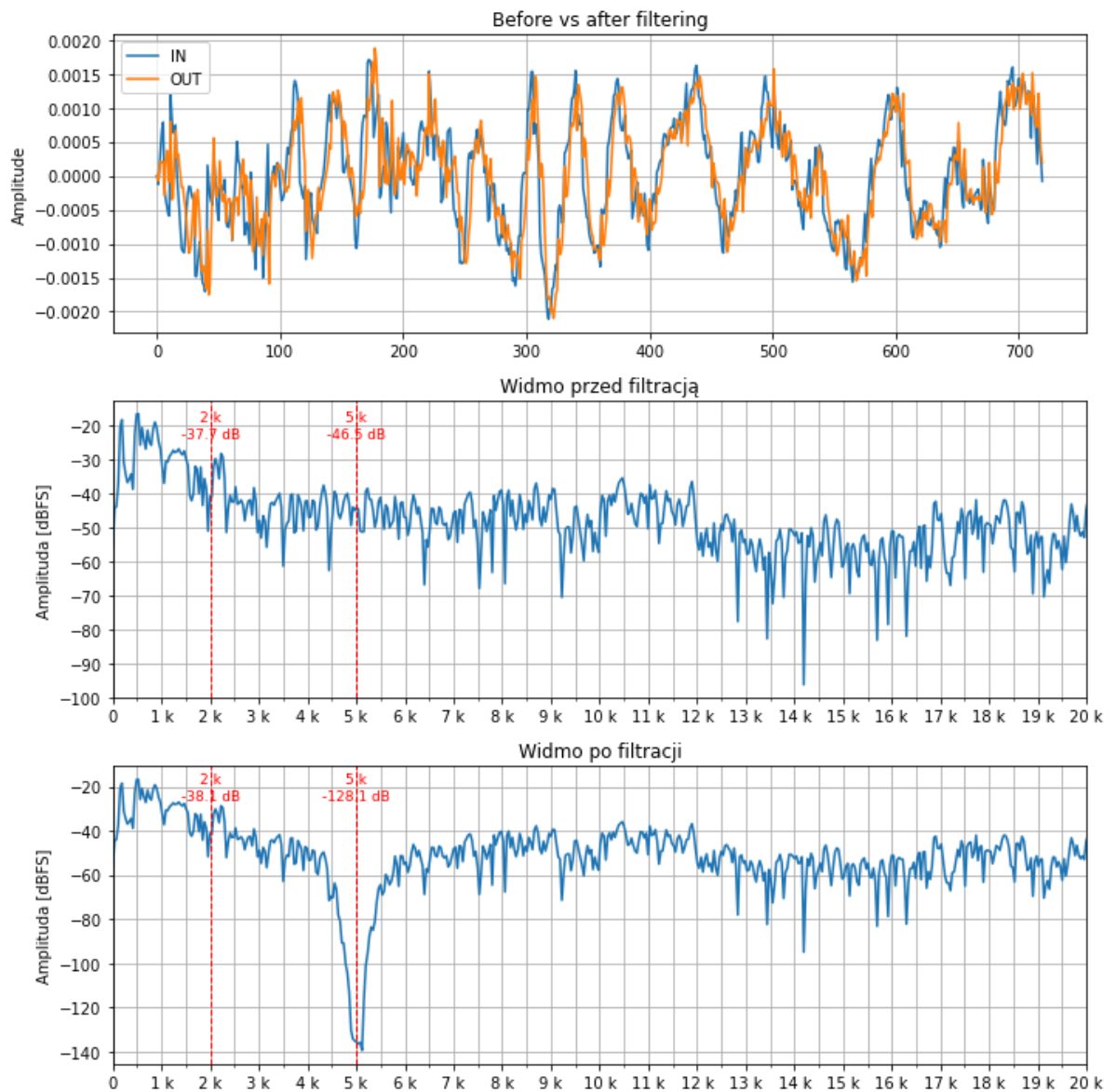
```
In [14]: single_sample_in = [1.0]
_, _, dt = filter_and_plot("filters/5k_atteunate_filters_narrow.json", single_sample_in)
print(f"Filtracja sygnału zakładając pipelining: {dt*1e3:.2f} ms")
```

Filtering time: 1.07 ms

Filtracja sygnału zakładając pipelining: 1.07 ms

Filtracja programowa sygnału wejściowego z DMA

```
In [15]: soft_out_t_domain, soft_out_f_domain, _ = filter_and_plot("filters/5k_atteunate_fil
```



Filtering time: 1.80 ms

Przeprowadzone testy

Test modułu I2S

Przeprowadzono test modułu sterującego I2S wykorzystując ILA taktowany zegarem 100 MHz

Test polegał na sprawdzeniu czy moduł generuje żadaną częstotliwość przez obserwację sygnałów zegarowych: `MCLK`, `BCLK`, `LRCK`. Zgodnie z dokumentacją modułu Pmod I2S2 aby osiągnąć częstotliwość próbkowania 48kHz zegary dostarczane do modułu powinny spełniać warunki przedstawione na rysunku poniżej:

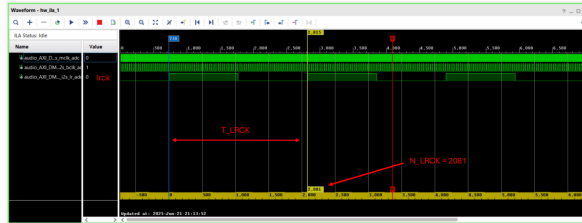
- częstotliwość `LRCK` = 48 kHz
- częstotliwość `MCLK` = 12.288 MHz
- częstotliwość `BCLK` = $\frac{f_{MCLK}}{4}$

	MCLK (MHz)									
LRCK (kHz)	64x	96x	128x	192x	256x	384x	512x	768x	1024x	1152x
32	-	-	-	-	8.1920	12.2880	-	-	32.7680	36.8640
44.1	-	-	-	-	11.2896	16.9344	22.5792	33.8680	45.1580	-
48	-	-	-	-	12.2880	18.4320	24.5760	36.8640	49.1520	-
64	-	-	8.1920	12.2880	-	-	32.7680	49.1520	-	-
88.2	-	-	11.2896	16.9344	22.5792	33.8680	-	-	-	-
96	-	-	12.2880	18.4320	24.5760	36.8640	-	-	-	-
128	8.1920	12.2880	-	-	32.7680	49.1520	-	-	-	-
176.4	11.2896	16.9344	22.5792	33.8680	-	-	-	-	-	-
192	12.2880	18.4320	24.5760	36.8640	-	-	-	-	-	-
Mode	QSM				DSM			SSM		

Table 1. Common Clock Frequencies

Konfiguracje pracy Pmod I2S2.

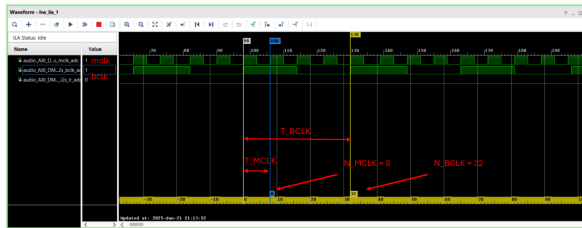
- Zegar LRCK



Test zegara LRCK.

$$f_{LRCK} = \frac{f_{ILA}}{N_{LRCK}} = \frac{100\,000\,000}{2081} = 48\,053\,[Hz] \approx 48\,000\,[Hz]$$

- Zegary MCLK i BCLK



Test zegarów MCLK i BCLK.

$$f_{MCLK} = \frac{f_{ILA}}{N_{MCLK}} = \frac{100\,000\,000}{8} = 12\,500\,000\,[Hz] \approx 12.288\,[MHz]$$

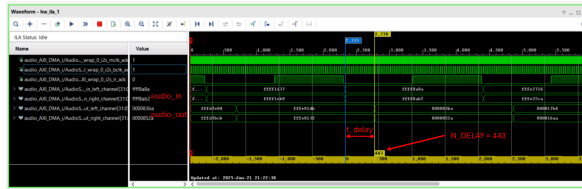
$$f_{BCLK} = \frac{f_{ILA}}{N_{BCLK}} = \frac{100\,000\,000}{32} = 3\,125\,000\,[Hz] \approx \frac{f_{MCLK}}{4}$$

Parametry:

- f_{ILA} – częstotliwość zegara taktującego moduł ILA
- f_{LRCK} – częstotliwość zegara LRCK
- f_{MCLK} – częstotliwość zegara MCLK
- f_{BCLK} – częstotliwość zegara BCLK
- N_{LRCK} – liczba taktów zegara f_{ILA} w okresie LRCK
- N_{MCLK} – liczba taktów zegara f_{ILA} w okresie MCLK
- N_{BCLK} – liczba taktów zegara f_{ILA} w okresie BCLK

Pomiar czasu filtracji

Kolejnym testem było wyznaczenie czasu filtracji przez obliczenie opóźnienia między próbkami wejściowymi a wyjściowymi. Opóźnienie to jest wprowadzane przez bank filtrów.



Test czasu filtracji.

$$t_{delay} = \frac{1}{\frac{f_{ILA}}{N_{DELAY}}} = \frac{1}{\frac{100\,000\,000}{443}} \approx \frac{1}{225\,734} \approx 4.43 [\mu s]$$

Parametry:

- f_{ILA} – częstotliwość zegara taktującego moduł ILA
- N_{DELAY} – liczba taktów zegara f_{ILA} w okresie `t_delay`

Porównanie filtracji sprzętowej z programową

W celach porównania filtracji sprzętowej i programowej przygotowano wyżej filtry o takich samych współczynnikach. Wyznaczono czas filtracji dla obu przypadków, sygnał błędu oraz inne parametry.

```
In [19]: x_t = left_in_f
x_f = signal_in_f_domain
hw_out_t = left_out_f
hw_out_f = hardware_out_f_domain
sw_out_t = soft_out_t_domain
sw_out_f = soft_out_f_domain

def bench_filters(x_t, x_f,
                  y_hw_t, y_hw_f,
                  y_sw_t, y_sw_f,
                  fs=48_000,
                  title="Filter-bench"):
    """
    Compare HW and SW filter outputs against the raw input.

    All *time* arrays are trimmed to the same minimum length.
    All *frequency* arrays are trimmed to the same minimum length (end-cut).

    Returns a dict with MSE / SNR / max error for HW, SW and their difference
    and shows six diagnostic plots.
    """

    # ----- 1. equalise time-domain lengths -----
    Nt = min(len(x_t), len(y_hw_t), len(y_sw_t))
    x_t, y_hw_t, y_sw_t = x_t[:Nt], y_hw_t[:Nt], y_sw_t[:Nt]

    # ----- 2. equalise frequency-domain lengths -----
    Nf = min(len(x_f), len(y_hw_f), len(y_sw_f))
    x_f, y_hw_f, y_sw_f = x_f[:Nf], y_hw_f[:Nf], y_sw_f[:Nf]

    # ----- 3. error (time) + metrics -----
    err_hw_t = y_hw_t - x_t
    err_sw_t = y_sw_t - x_t
    err_diff_t = err_sw_t - err_hw_t

    def metrics(ref, err):
        mse = float(np.mean(err**2))
        snr = 10 * np.log10(np.mean(ref**2) / mse + 1e-30)
```

```

        mabs = float(np.max(np.abs(err)))
        return mse, snr, mabs

mse_hw, snr_hw, max_hw = metrics(x_t, err_hw_t)
mse_sw, snr_sw, max_sw = metrics(x_t, err_sw_t)
mse_df, snr_df, max_df = metrics(err_hw_t, err_diff_t)

# ----- 4. error (frequency) -----
err_hw_f = y_hw_f - x_f
err_sw_f = y_sw_f - x_f
err_diff_f = err_sw_f - err_hw_f
mag = lambda z: 20 * np.log10(np.abs(z) + 1e-12)
f_hz = np.fft.rfftfreq(Nt, 1 / fs)[:Nf] # align with trimmed spectra

# ----- 5. plots -----
fig, axs = plt.subplots(3, 2, figsize=(12, 9))

# 5a time-domain errors (first 2 000 samples)
axs[0, 0].plot(err_hw_t[:2000], label="HW error")
axs[0, 0].plot(err_sw_t[:2000], label="SW error", alpha=.7)
axs[0, 0].set_title(f"{title} - error, first 2 000 samples")
axs[0, 0].grid(); axs[0, 0].legend()

# 5b difference of errors (time)
axs[1, 0].plot(err_diff_t[:2000])
axs[1, 0].set_title("Difference of errors (SW - HW)")
axs[1, 0].grid()

# 5c full-frame errors overlay
axs[2, 0].plot(err_hw_t, alpha=.6, label="HW err")
axs[2, 0].plot(err_sw_t, alpha=.6, label="SW err")
axs[2, 0].set_title("Errors - full frame")
axs[2, 0].set_xlabel("Sample"); axs[2, 0].grid(); axs[2, 0].legend()

# 5d spectra of errors
axs[0, 1].semilogx(f_hz, mag(err_hw_f), label="HW error")
axs[0, 1].semilogx(f_hz, mag(err_sw_f), label="SW error", alpha=.7)
axs[0, 1].set_title("Error spectra (dBFS)")
axs[0, 1].grid(which="both"); axs[0, 1].legend()

# 5e spectrum of error difference
axs[1, 1].semilogx(f_hz, mag(err_diff_f))
axs[1, 1].set_title("Spectrum of error difference"); axs[1, 1].grid(which="both")

# 5f spectra of signals
axs[2, 1].semilogx(f_hz, mag(x_f), label="Input")
axs[2, 1].semilogx(f_hz, mag(y_hw_f), label="HW out", alpha=.7)
axs[2, 1].semilogx(f_hz, mag(y_sw_f), label="SW out", alpha=.7)
axs[2, 1].set_title("Spectra of input & outputs (dBFS)")
axs[2, 1].set_xlabel("Frequency [Hz]")
axs[2, 1].grid(which="both"); axs[2, 1].legend()

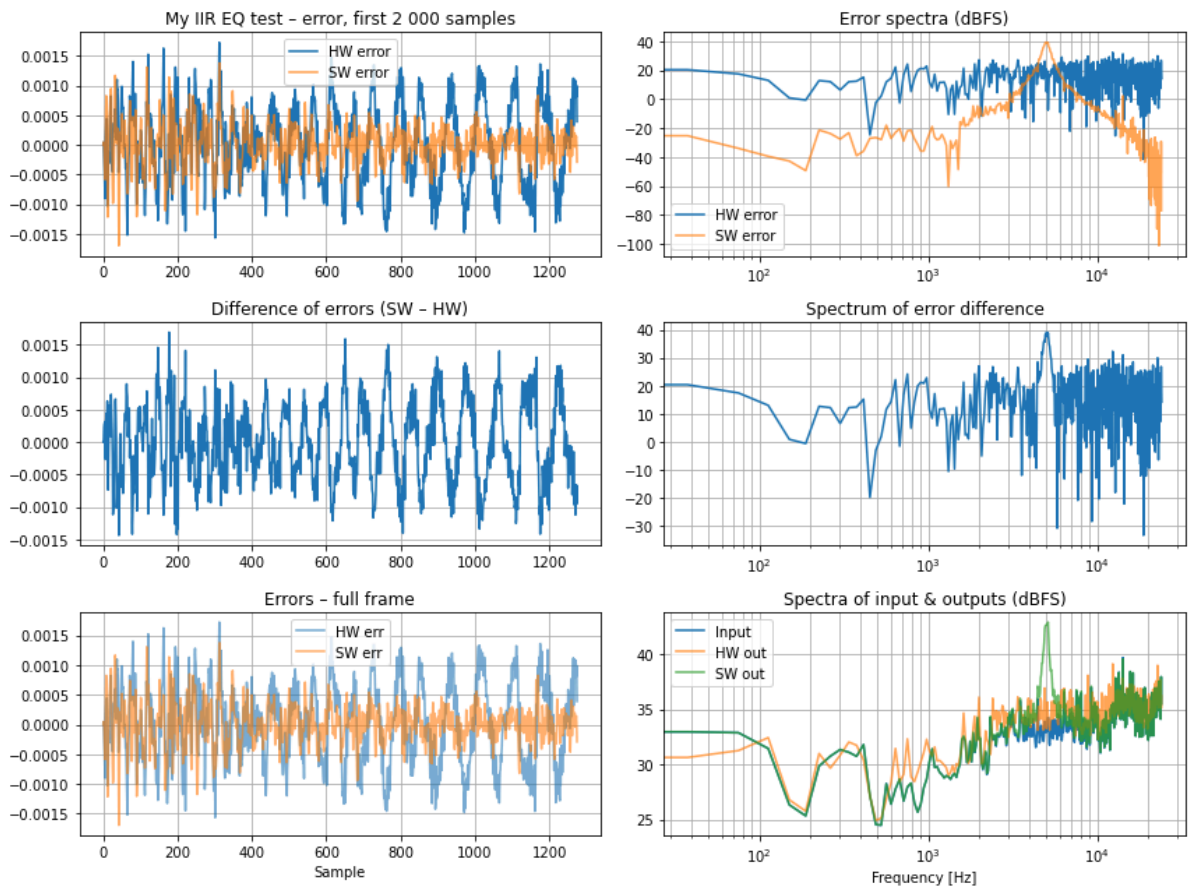
plt.tight_layout()
plt.show()

return {
    "mse_hw": mse_hw, "snr_hw": snr_hw, "max_hw": max_hw,
    "mse_sw": mse_sw, "snr_sw": snr_sw, "max_sw": max_sw,
    "mse_diff": mse_df, "snr_diff": snr_df, "max_diff": max_df,
}

results = bench_filters(x_t, x_f,
                        hw_out_t, hw_out_f,
                        sw_out_t, sw_out_f,

```

```
fs=48_000,
title="My IIR EQ test")
```



```
In [26]: for key, value in results.items():
          print(f"{key} | {value}")
```

```
mse_hw | 4.4841570522917157e-07
snr_hw | 0.780760153143591
max_hw | 0.0017243027687072754
mse_sw | 1.1999271836829288e-07
snr_sw | 6.506019380095898
max_sw | 0.00169431606100145
mse_diff | 3.808240736832891e-07
snr_diff | 0.7095642045951172
max_diff | 0.0016922866780472259
```

Parametr	SW	HW	Różnica (SW - HW)
mse	1.1999271836829288e-07	4.4841570522917157e-07	-3.284229868608787e-07
snr	6.506019380095898	0.780760153143591	5.725259226952307
max	0.00169431606100145	0.0017243027687072754	-2.9986707705825395e-05

Czas filtracji sprzętowej: 4.43 $[\mu s]$

Czas filtracji programowej: 1.07 $[ms]$

Czas filtracji sprzętowej jest znacznie szybszy (kilkaset razy) od przykładowej implementacji sprzętowej

Bibliografia:

- [YetAnotherElectronicsChannel. „FPGA-Audio-IIR“.](#)
- [STMicroelectronics. „BQD Filter Design Equations“.](#)
- [Pmod I2S2](#)

In []: